The Final Plan: Co-Dispatch

The framing (why this wins, in one paragraph)

The Trucker Path deck literally spells out the winning formula in three places. First, the challenge says *"reducing cost per mile and driver downtime"* — so every feature has to be measurable in dollars or minutes saved. Second, the "what good looks like" slide says *"shows the intelligence, not just the data — decisions, not dashboards"* — so we never show a screen that's just displaying info, every pixel is a decision being made. And third, the gap slide literally calls the problem *"two good halves glued together with phone calls"* — so our one-line pitch is just quoting that back at them: *"Trucker Path already has the inputs. We're the layer that turns them into decisions."* Most teams won't read the deck this carefully. We'll show up with a product that mirrors their own language, which makes the judges feel like we actually understood the brief.

What we're building: Co-Dispatch

Co-Dispatch is a co-pilot that lives alongside NavPro, not a replacement. Trucker Path already nailed the driver side — John has truck-safe navigation, parking, fuel, and an HOS clock ticking on his phone. The dispatcher side has the Command web portal with maps and document capture. What's missing is the layer between them that turns those inputs into decisions, which is exactly the gap the Trucker Path deck identifies with the quote *"the product has the inputs. It doesn't yet have the layer that turns them into decisions."* We are that layer.

The product does four things end-to-end, and each one kills a specific moment in Maria's day as shown in their own "Maria's day" walkthrough slides.

Workflow 1 — Morning triage at 7am

The Trucker Path deck shows Maria at 7am with the caption *"she still texts 20 drivers every morning"* and calls out three specific gaps: no unified view of who's ready, HOS remaining is phone-and-text, and triage takes the first 90 minutes of her day. That's the problem we attack first.

We replace all of it with a single screen. On page load the AI calls the NavPro driver query endpoint, then the driver performance endpoint for each active driver, then the dispatch endpoint to check active trips, and finally synthesizes the whole fleet into a morning brief. Something like *"14 drivers ready to run, 3 need rest within 2 hours, 2 have compliance flags —*

Mike's inspection expires Friday, Sara's on a fatigue pattern — 1 truck is in maintenance." One scroll, zero texts sent, 90 minutes saved. This is Stop 1 of Maria's day gone.

Workflow 2 — Explainable load assignment at 9am

The 9am slide in their deck shows Maria staring at a load and has three gap bullets: *"no optimizer scoring drivers by HOS left, position, cost, and fit"*, *"wrong driver → costly deadhead + ripple into tomorrow's board"*, and *"no check on compliance risk before assigning."* Our Workflow 2 answers all three point by point.

A new load comes in — Maria pastes an email, drags a PDF rate confirmation, or types it in. The AI parses it into structured data (origin, destination, weight, commodity, pickup window, rate). Then it queries every available driver and produces a ranked table with full math visible — deadhead miles to pickup based on the driver's actual GPS trail from the tracking endpoint, HOS hours remaining versus hours needed for the trip, estimated fuel cost on the optimized route, ETA confidence, and the ripple impact on tomorrow's already-planned loads. Every number is clickable — click "deadhead 42 miles" and it shows the route from GPS point to pickup; click "HOS check" and it shows the calculation; ask "why not Jake?" and the AI explains the counterfactual using the same math.

One click dispatches through the NavPro trip creation endpoint. This is our core demo moment because it's literally the phrase from the judges' brief — *"intelligence, not just data."*

Workflow 3 — Backhaul pairing (the winning move)

This is the feature that beats 19 other teams and I keep coming back to it because none of them will think of it. Every other team will stop at "who gets this load." We go one level up: when Maria dispatches a load, the AI simultaneously searches broker marketplaces for return loads that intersect the route home.

So for the demo, load comes in PHX → SFO at $3,200. Mike wins the ranking. But before Maria clicks confirm, the AI says *"Hold on — Mike will deadhead 340 miles back to Phoenix. I searched TruckLoads along this corridor and found 3 backhaul options. Best pick is SFO to Vegas to Phoenix at $2,400 with only 85 total deadhead miles. Round-trip profit goes from $2,100 to $4,800."* One click books both loads through NavPro's trip creation endpoint. Mike gets a full round-trip plan, Maria just doubled the revenue on this truck for the week.

This wins because of four reasons layered on top of each other. First, it directly attacks the judging criterion — *"reducing cost per mile and driver downtime"* — because deadhead miles are literally cost per mile and backhaul fills the empty return. Second, it's genuinely novel in a live-API, real-time-decision context; most fleets do backhaul matching manually on DAT or not at all, and doing it at the moment of dispatch with full HOS and driver context is the thing that makes a Trucker Path product person realize "this is a feature we could ship." Third, it naturally upsells Trucker Path's own TruckLoads marketplace (the 150K loads/day product they already own), which means the Trucker Path judges see revenue alignment for their own business, not

just a clever AI feature. And fourth, it's the most visual demo moment — a before/after comparison of round-trip profit that judges will literally photograph with their phones.

Workflow 4 — Proactive monitoring with voice

The in-transit slide in their deck shows the gap as *"loads in transit go dark until something breaks — no alert for route deviation, long idle, or weather slowdown. Maria finds out from the customer, not the system."* We don't just alert — we draft the fix and deliver it by voice.

Every 30 seconds the backend polls the NavPro driver dispatch endpoint for every active trip. We compare the actual GPS trail against the scheduled route and watch for four things — route deviation beyond a tolerance, long idle (truck stopped more than 30 min outside a known rest stop), HOS running out mid-trip, and ETA slip. When any of those trip, the AI doesn't just send an alert. It drafts the complete intervention package — customer SMS with a revised ETA, nearest-driver relay plan using the same scoring engine as Workflow 2, and a reroute suggestion through NavPro if traffic is the cause.

The emotional demo twist is that Maria doesn't have to be looking at the screen. Her phone rings with a real ElevenLabs voice saying *"Maria, truck 14 has been stopped 47 minutes outside Barstow. Engine idle pattern suggests breakdown. I've drafted the customer SMS, found relay driver Kevin 28 miles away with 9 HOS hours left. Say 'execute' to approve."* She says execute. The screen updates live — SMS sent, relay dispatched, reroute pushed. This is the emotional peak of the demo, and it also wins us the ElevenLabs side track as a bonus.

What we deliberately do NOT build

This list is almost as important as the build list because scope creep is how hackathon teams die. We don't build a driver-side app — John already has one and NavPro covers it. We don't build generic chat — the AI only runs these four workflows, because a chatbot that does everything will fail at something on stage. We don't build auth, settings, user management, or light mode. We don't build invoicing even though it's the fifth gap on their slide — it would dilute the demo and the backhaul moment hits way harder. If somebody has genuine spare cycles at the end we can make auto-invoice a single bonus slide in the deck showing "here's the next feature" and nothing more. We don't build real broker integrations — we mock a realistic load board with 500 seeded loads and structure the code so it's swappable for DAT or Truckstop later by replacing one module. We don't build mobile anything — Maria is on desktop, John is on NavPro's existing mobile app.

If any team member starts building anything on this list, the other two stop them. This is the single rule that wins hackathons.

The tech stack (all free, no paid APIs)

Everything here is either free forever or free for the duration of the hackathon. Total monthly cost: $0.

The LLM — Groq Cloud (free tier). Gives us Llama 3.3 70B and Gemma 2 9B at 30 requests per minute, sub-second latency (which matters for voice). Llama 3.3 70B supports native tool use, which is what the agent loop needs. Sneaky win — we're literally running open models, which makes our "LLM-agnostic, run on-prem for enterprise" GTM story actually provable, not architecture-deck handwaving. Teams that hardcoded GPT-4 can't pitch this.

Backup LLM — Google Gemini 2.0 Flash (free tier). 15 requests per minute, 1,500 per day, has vision (for parsing load PDFs and rate confirmation images), supports function calling. We use it specifically for the load parsing step because of vision, and as a fallback if Groq rate-limits during the demo.

Voice — ElevenLabs Creator tier (free for 1 month via the hackathon). Claim it through the Discord coupon-codes channel using the hackathon registration email. The PDF says emails are pre-loaded 72 hours before the event, so claim this TODAY or we lose the voice angle.

Frontend + backend — Next.js 14 with App Router + TypeScript. Everything in one repo. One deploy. No merge hell.

UI — Tailwind + shadcn/ui. Dark theme only, ops-terminal aesthetic, installs in one command.

Map — Mapbox GL JS. Free tier is 50K map loads per month. A real map is non-negotiable for credibility — judges lose trust the moment they see a static image with CSS pins.

Database — SQLite via Prisma. Zero config, file-based, free, stores decision log, load queue, assignments.

Broker mock — seeded JSON file. 500 realistic US loads with origins, destinations, rates, pickup windows. One TypeScript function does haversine corridor search. Zero external dependencies.

Deployment — Vercel free tier. One URL, auto-deploys on git push.

Environment variables needed: `GROQ_API_KEY`, `GEMINI_API_KEY`, `ELEVENLABS_API_KEY`, `MAPBOX_ACCESS_TOKEN`, NavPro credentials. All free.

Why not split frontend/backend: costs us a day in deployment and merge conflicts, and for three CS majors using AI coding agents, keeping everything in TypeScript means the agents have one mental model.

Team split — three parallel tracks, minimal overlap

Three CS majors, each with an AI coding agent (Cursor, Continue, Aider, Cline — whatever's free and local works). The key is that nobody touches anybody else's files, because AI agents love to "helpfully" rewrite code they don't need to touch.

Person A — The Data Spine

Owns all data in and out. Nobody else touches the data layer — if anyone needs drivers or loads they call A's endpoints. A is also the one who goes and gets NavPro demo credentials the moment the hackathon opens, because that's the long pole that blocks everybody.

Day 1 tasks for A's AI agent, in order:

- Log into the NavPro demo account and invite 15-20 seed drivers spread across Arizona, California, and Nevada (good corridor for the backhaul demo). Give them realistic names and phone numbers
- Create `lib/navpro.ts` with typed TypeScript wrappers for every endpoint we need: `queryDrivers`, `getDriverPerformance`, `getDriverDispatch` (for GPS trails), `createTrip`, `queryTrips`, `queryPOI`. Each wrapper returns responses typed against the schemas in `api-1.json`
- Create `lib/loadBroker.ts` — the mock TruckLoads marketplace. Seed it with 500 loads covering the US with a bias toward the PHX/LA/SFO/Vegas corridor. Expose `searchLoadsNearRoute(origin, destination, pickupWindow)` using haversine distance to filter loads within 50 miles of the corridor
- Create `prisma/schema.prisma` with models for `DecisionLog`, `LoadAssignment`, `ActiveTrip`, `InterventionDraft`
- Create `app/api/fleet/snapshot/route.ts` — one mega-endpoint that returns drivers with HOS, active trips, pending loads, compliance flags, everything the UI needs in one call
- Create `app/api/dev/simulate/route.ts` — a hidden dev endpoint that triggers fake breakdowns for the Act 3 demo. Nobody mentions this in the demo; it just runs in the background

Person B — The Brain

Owns the AI layer, scoring math, and voice. Zero UI code. Pure business logic that can be tested with curl before C wires it to the frontend.

Day 1 tasks for B's AI agent, in order:

- Create `lib/llm.ts` — the adapter layer. Exposes `callLLM(messages, tools)` with a pluggable backend. Default is Groq (Llama 3.3 70B), fallback is Gemini 2.0 Flash. This is the file that makes our "LLM-agnostic" pitch actually real
- Create `lib/scoring.ts` — pure function `scoreDriverForLoad(driver, load, fleetState)` that returns `{score, deadheadMiles, hosCheck, fuelCost, etaConfidence, rippleImpact, rationale}`. All pure math, no LLM call — the LLM formats the output but doesn't do the numbers. This matters because judges will ask "is the ranking done by AI or real math" and the answer is "real math, explainable by AI"
- Create `lib/backhaul.ts` — the centerpiece. `findBackhauls(outboundLoad, driverHomeBase, returnWindow)` calls `loadBroker.searchLoadsNearRoute` and returns ranked round-trip options. Include round-trip profit calculation and HOS feasibility for the return leg
- Create `lib/agent.ts` — the Groq-powered tool-use agent with exactly 5 tools: `get_fleet_snapshot`, `score_assignment`, `find_backhauls`, `monitor_trips`, `draft_intervention`. The system prompt is tight — it only does dispatcher workflows, it refuses off-topic questions, it always shows math
- Create `lib/voice.ts` — ElevenLabs integration. Two functions: `speakAlert(text)` uses TTS for inbound calls; `listenForCommand()` uses their conversational agent for outbound commands. Default voice is a professional American female for Maria's dispatcher assistant
- Create `app/api/agent/route.ts` — streaming endpoint that runs the agent loop and streams tokens back to the UI

Person C — The Face

Owns everything the judges see. No backend code, no API calls beyond calling A and B's endpoints. C is also the best candidate to be the primary demo presenter since they'll know the UI cold.

Day 1 tasks for C's AI agent, in order:

- Set up Next.js 14 with Tailwind and shadcn/ui in the repo. Dark theme tokens locked, no light mode ever
- Build the three-panel main layout in `app/page.tsx`: loads queue left 25%, fleet map center 50%, AI copilot pane right 25%
- Build `components/LoadInbox.tsx` — the paste/drop zone for new loads, with file upload for PDFs and a textarea for email paste

- Build `components/FleetMap.tsx` — Mapbox component with driver pins colored by HOS status (green = fresh, yellow = low, red = must-rest) and active trip routes drawn as lines
- Build `components/DriverRankCard.tsx` — the Bloomberg-terminal dense ranking card. Driver name, score, deadhead miles, HOS, fuel cost, ETA, ripple. Expandable "why" section that shows the math. This component has to look insanely good
- Build `components/BackhaulModal.tsx` — the money shot. Round-trip visualization showing outbound + return, profit comparison bar, one-click accept. This is the component that wins the hackathon so it gets the most polish
- Build `components/VoiceAlert.tsx` — popup that plays the ElevenLabs audio with a waveform visualization and approve/reject buttons
- Build `components/DecisionLog.tsx` — timeline of every AI action for the closer scene. Each entry shows timestamp, action, math, outcome

Day-by-day timeline

Assuming a 48-hour hackathon (adjust to your actual clock). I'm building in buffer because something always breaks.

Day 1, hours 0 to 2 — Setup. All three set up the monorepo, install deps, deploy a "hello world" to Vercel. A gets NavPro credentials and starts seeding drivers. B claims the ElevenLabs code via Discord. C sets up Tailwind and the shadcn components. No features yet — just confirm deployment works. Also lock the decision: Next.js + TypeScript + Groq. After this hour, nobody is allowed to re-open the stack debate.

Day 1, hours 2 to 6 — Foundation. A finishes NavPro wrappers and seeds all demo data. B gets the Groq agent loop working with one mock tool returning hardcoded JSON (just to confirm tool use works). C builds the static three-panel layout with placeholder data. By hour 6 there's a deployed URL that looks like the product even though nothing's connected yet.

Day 1, hours 6 to 12 — End-to-end spine. A finishes the load broker mock. B gets `score_assignment` working end-to-end with A's real NavPro data — you can curl the agent endpoint with a load JSON and get back a real ranked driver list. C wires the UI flow from "paste load" to "AI responds" to "ranking displays." By end of hour 12 we have Workflow 2 working end-to-end with real NavPro data. This is the minimum viable demo — if everything else fails from here, we still have something to show.

Day 1, hours 12 to 16 — The backhaul moment. All three converge on Workflow 3. B builds `findBackhauls`. A enriches the load broker with smarter corridor queries. C builds the `BackhaulModal` component and wires the flow so that after the driver ranking appears, the modal slides in with the backhaul options. This has to work perfectly before anyone sleeps because it's the centerpiece. Rehearse Workflow 2+3 end-to-end at least once before bed.

Sleep. Don't skip this. A broken demo from tired presenters loses to a clean demo from rested ones.

Day 2, hours 0 to 6 — Voice and monitoring. B integrates ElevenLabs for both inbound (TTS alert) and outbound (conversational dispatch command). B also builds the monitoring polling loop that checks all active trips every 30 seconds and triggers interventions. C builds `VoiceAlert.tsx` with audio playback and waveform. A builds the hidden dev-mode "simulate breakdown" button that flips a trip's state to trigger the alert. By hour 6, Workflow 4 works end-to-end.

Day 2, hours 6 to 10 — Polish and rehearse. All three switch to demo mode. Demo script written word-for-word in a shared doc. Rehearsed at least 5 times with someone playing a judge trying to break it. Seed data tweaked so the numbers on screen look impressive ($5,694 saved sounds better than $347). C builds the decision log closer tab. Backup demo video recorded in OBS with narration at the end so if the live demo crashes we have a fallback to play.

Day 2, hours 10 to 12 — Deck, GTM, submit. Five-slide deck built in Google Slides or Figma. GTM one-pager written. Devpost submission in with working URL and backup video. ElevenLabs showcase submission in. Shower. Eat. Sleep if possible.

The three-act demo script (the whole thing is built toward this)

Every line of code we write for the next 48 hours is judged against one question: *does this make one of these three scenes land harder?* If yes, build it. If no, don't.

Act 1 — 7am triage (30 seconds). Presenter opens the deployed URL. The screen already shows the morning brief because the AI ran on page load. Narration: *"This is Maria's 7am. Normally she's texting 20 drivers to figure out who can run today. Co-Dispatch did that in one API call. 14 drivers ready, 3 need rest, 2 compliance flags, Sara has a fatigue pattern I'll get to later. This is Stop 1 of her day, done."*

Act 2 — 9am load + backhaul (90 seconds, centerpiece). Presenter pastes a realistic broker email into the load inbox. Parse happens on-screen in 3 seconds. Fleet snapshot appears with ranked drivers. Narration: *"New load, Phoenix to San Francisco, $3,200. Here's the ranking — Mike wins on deadhead because he's already in Tempe, HOS is clean, ripple impact minimal. Click the score — full math visible. Why not Jake? He's 4 HOS hours short of the required drive time. Alright, dispatching Mike —"* [click assign, modal slides in] *"— wait. Before Maria confirms, our AI already pulled 3 backhaul options from TruckLoads along this corridor. Best pick — San Francisco to Vegas to Phoenix, $2,400, only 85 total deadhead miles. Round-trip profit jumps from $2,100 to $4,800. One click, full circle dispatched through the NavPro trip API."*

Act 3 — 2pm intervention with voice (60 seconds, emotional moment). Presenter clicks "next scene" or lets the simulated breakdown trigger. Phone rings through laptop speakers. ElevenLabs voice plays: *"Maria, truck 14 has been stopped 47 minutes outside Barstow. Engine idle pattern suggests breakdown. I've drafted a customer SMS with a 3-hour delay, found relay driver Kevin 28 miles away with 9 HOS hours left, and prepped the swap plan. Say 'execute' to approve."* Presenter says "execute." Screen updates in real time showing SMS drafted, relay plan confirmed, reroute pushed to the original driver's NavPro app.

Closer — 15 seconds. Presenter flips to the decision log tab. Narration: *"Every one of these decisions was logged with its full math. This month: 2,847 deadhead miles eliminated via backhaul pairing, $5,694 in revenue recovered, 4.2 hours saved for Maria every day. Notice we haven't built a Cost Intelligence feature — we didn't have to. It's what happens automatically when every decision is transparent."* That last sentence is the mic drop — it implicitly claims three of the five gaps (Smart Dispatch, Proactive Alerts, Cost Intelligence) while we only explicitly built two, which is exactly what the brief asks for.

The 5-slide deck (use the source slides' own language against them)

Slide 1 — The Problem. Title: *"Two good halves, glued together with phone calls."* This is a direct quote from their deck. One image showing Maria's desk with a ringing phone and a wall of unread texts. Subtitle: *"The product has the inputs. It doesn't yet have the layer that turns them into decisions."* (Also their words.)

Slide 2 — The Product. Title: *"Co-Dispatch — decisions, not dashboards."* (Their phrase again.) One big screenshot of our three-panel dashboard with the backhaul modal open. Three tiny captions pointing at the morning brief, the ranked assignment, and the backhaul recovery. That one image should sell the whole product before they read anything.

Slide 3 — Architecture. Clean diagram flowing left to right. NavPro API on the left (drivers, tracking, trips, POIs). Our agent layer in the middle with the five tools labeled. Decision log underneath the agent showing every action flowing to persistence. UI and voice on the right. Critically, the LLM box in the middle shows *"Groq (Llama 3.3) | Gemini Flash | Local Gemma on-prem"* with arrows showing it's pluggable. This is the slide that sells the enterprise GTM story.

Slide 4 — Why we win. Four-box grid, one sentence per box:

1. *Backhaul pairing — every dispatch becomes a round-trip.*
2. *Voice co-dispatcher — Maria doesn't have to be at the screen.*
3. *Explainable math — every number clickable, every ranking defensible.*
4. *LLM-agnostic — cloud for small fleets, on-prem for enterprise.*

Slide 5 — GTM. Title: *"A price model that aligns with the customer's pocket."*

- Base: $29 per truck per month (cheap enough that a 20-truck fleet says yes without a second meeting).
- Variable: 2% of backhaul revenue we recover for them (only pay more when we make them more).
- Enterprise: custom pricing with on-prem Llama/Gemma for data-compliance-sensitive fleets like healthcare freight, DOD contractors, Canadian cross-border.
- Acquisition: add-on SKU inside the existing NavPro sales motion. Self-serve toggle in Command web portal. Upsells TruckLoads as primary broker integration.
- Unit economics: if we recover 100 deadhead miles per truck per month we've paid for the base subscription. We're recovering 500+ in the demo.

Why we beat 30* other teams (the specific traps other teams fall into)

Most teams are going to fail for one of six reasons, and we're set up to avoid each one.

Some teams will build a pretty dashboard with a chat box. We beat them because the judges literally wrote *"decisions, not dashboards"* in the brief, and every screen we show is a decision being made or one being approved.

Some teams will have the AI say "Driver Mike is best" with no justification. Black-box vibes, instantly loses credibility when a judge pokes it. We beat them with explainable math — every number is a pure function, every ranking has a rationale, every "why not X?" has a real answer.

Some teams will stop at load assignment. We beat them on backhaul. This is genuinely novel in the live-API, at-the-moment-of-dispatch context, and it directly attacks the cost-per-mile criterion on their rubric. Backhaul is the single biggest reason we win.

Some teams will mock all their data. We beat them by calling the real NavPro API live on stage. The API went live March 31st, 2026 — it's a few weeks old, most teams won't even have seeded their demo account. We seed ours on Day 1 hour 1 and every demo call is real. The moment we say "this is a real NavPro response" the room tilts.

Some teams will skip voice or slap on ElevenLabs as an afterthought. We beat them with a voice integration that's actually woven into the workflow — the phone ringing during Act 3 is the moment the demo becomes memorable, not a gimmick.

And finally, some teams will ad-lib their demo and fumble it. We rehearse the three-act script at least 5 times and we have a backup video if the live system breaks. This is the most boring differentiator and also the most reliable one.

Free resources checklist (do this before coding starts)

Before anyone writes code, lock all of these:

- NavPro demo credentials — ask Trucker Path immediately at the hackathon kickoff
- Groq Cloud API key — free signup at groq.com, takes 30 seconds, keep it in `.env.local`
- Gemini API key — free from Google AI Studio, 60 seconds, backup LLM
- ElevenLabs Creator tier — free for 1 month via the hackathon Discord (must claim 72 hours before per the PDF, check this TODAY)
- Mapbox access token — free account, free tier, 60 seconds
- Vercel account — free, connect to GitHub repo
- GitHub repo — one shared repo, three branches, protected main
- Shared Notion/Google Doc for the demo script, the deck, and the GTM one-pager

Risks and how we mitigate them

I'd rather think about what can go wrong now than discover it during the demo.

The biggest risk is that Groq rate-limits us during the demo. 30 requests per minute sounds fine but voice + dispatch can burst. Mitigation: we cache every agent response for the demo flow so replays don't hit the API, and Gemini is wired up as instant fallback in `lib/llm.ts`.

The second risk is NavPro API issues — demo account permissions, rate limits, or endpoint downtime. Mitigation: A caches every API response to a local JSON file on Day 1, and we have a "offline demo" mode that plays back the cached responses if anything goes sideways. We still call live in the demo but fallback is seamless.

The third risk is the backhaul modal looking cheap. This is the moment that wins the whole thing, so C spends disproportionate time on it. If the round-trip profit comparison doesn't look amazing, the demo is 50% less impactful. Budget extra polish hours here.

The fourth risk is voice latency or ElevenLabs glitching mid-demo. Mitigation: we pre-generate the Act 3 alert audio at the start of the day and play the cached MP3 if the live call fails. The conversational agent piece (Maria saying "execute") can fall back to a button click if voice recognition fumbles.

The fifth risk is demo anxiety. All three present, so rehearse with the exact lines written out, not improvised. 5 rehearsals minimum.

What I need from the team now

Three things, in order, before any code gets written:

First, someone go get NavPro demo credentials. Right now, today, the moment you can. This is the single blocking dependency.

Second, confirm the stack — Next.js 14 TypeScript monorepo, Groq as primary LLM, Gemini as backup. Everyone agrees or pushes back right now, not at hour 10.

Third, claim the ElevenLabs free tier through the hackathon Discord coupon-codes channel. The PDF says emails are pre-loaded 72 hours before the event, so if you haven't done this, do it now or we lose the voice angle.

Once all three are checked, tell me and I'll write the exact file-by-file prompts you'll feed into your AI coding agents for each person's first session — with function signatures and type definitions locked so the three parallel tracks don't generate incompatible code.

Want me to save this whole plan as a markdown file you can drop into your repo's README or your team's Notion? Say the word and I'll create it.